PATENT REVIEW

Hierarchical Bandit-Guided Scientific Discovery

Language models generate programs that construct scientific solutions. Evaluated outcomes guide search and improve the generating model.

Technical solution and observed experimental evidence

HUAWEI

Contents

- Idea Compliance Check
- Background
  - LLM-Based Scientific Discovery
  - Prior Art: Candidate Sampling and Search
  - Technical Problems in Existing Approaches
- Technical Solution
  - Hierarchical Discovery Pipeline
  - Allocation and Policy Training
  - Dependency-Aware Execution
- Distinguishing Features of the Idea
- Experiments and Design Rationale
- Advantageous Effects of the Idea
- Value Statement for Obtaining a Patent

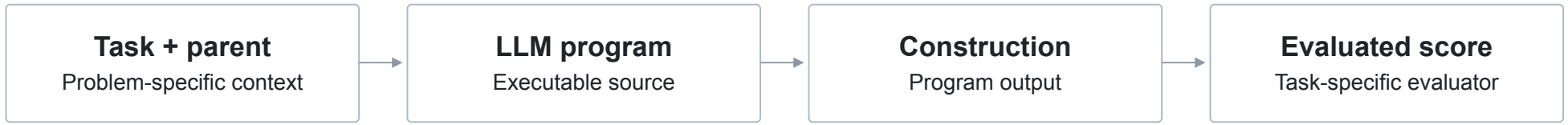
Idea Compliance Self-Check

No.	Self-check item	Declaration
1	Does the idea involve another party's non-public information from technical collaborations, procurement, supplier or customer communications, or another organization?	No
2	No non-public software algorithms, designs, layouts, or confidential technical solutions of a third party are used without authorization.	Confirmed
3	No third-party non-public information forms the basis of an improvement to the technology described in that information.	Confirmed
4	No non-public third-party information obtained by disassembly or reverse engineering is used in the patent application.	Confirmed
5	The idea does not involve non-public information of a former employer or work products belonging to that employer.	Confirmed

Inventor declarations. Use the applicable official submission checklist.

Background: LLM-Based Scientific Discovery

A large language model (LLM) writes a program that constructs a candidate solution. A task-specific evaluator checks the construction and assigns its score.



Search with a fixed model

The search reuses promising solutions as context for new programs. The model's weights stay unchanged.

Search with reinforcement learning

Evaluation also supplies a learning signal. The LLM updates its weights to favor programs with stronger outcomes.

Parent: a scored starting solution. **Rollout:** one generated program. **Archive:** retained solutions and their ancestry.

Success means finding a better valid construction, even if many attempts fail.

Prior Art: Candidate Sampling and Search

1. **Uniform sampling:** generate the same number of programs from each selected parent, regardless of early results.
2. **Evolutionary and island-based search:** populations retain and mutate strong evaluated candidates. Separate populations help maintain diversity.
3. **Tree-based search:** select a starting solution by balancing observed quality with exploration. UCT illustrates this tradeoff:

$$U(s) = Q(s) + c\sqrt{\frac{\log N}{1 + n(s)}}$$

s: archived state Q(s): observed quality n(s): expansions of s
N: total expansions c: exploration strength U(s): parent-selection score

Parent selection chooses a starting solution. The LLM then generates candidate programs from that context.

Illustrative UCT form. The PUCT parent selector also weights exploration using a prior P(s) over archived states.

Technical Problems in Existing Approaches

01 **One generation combines algorithm selection and implementation**

The LLM chooses an approach while writing the program. Additional samples can repeat the same approach without explicitly testing alternatives.

02 **Reinforcement learning can favor copies of an existing solution**

Reproducing an existing construction can still earn a high reward. Reinforcing these outputs can make the LLM generate more copies and fewer distinct constructions.

03 **A fixed sampling split cannot adapt to early evaluation results**

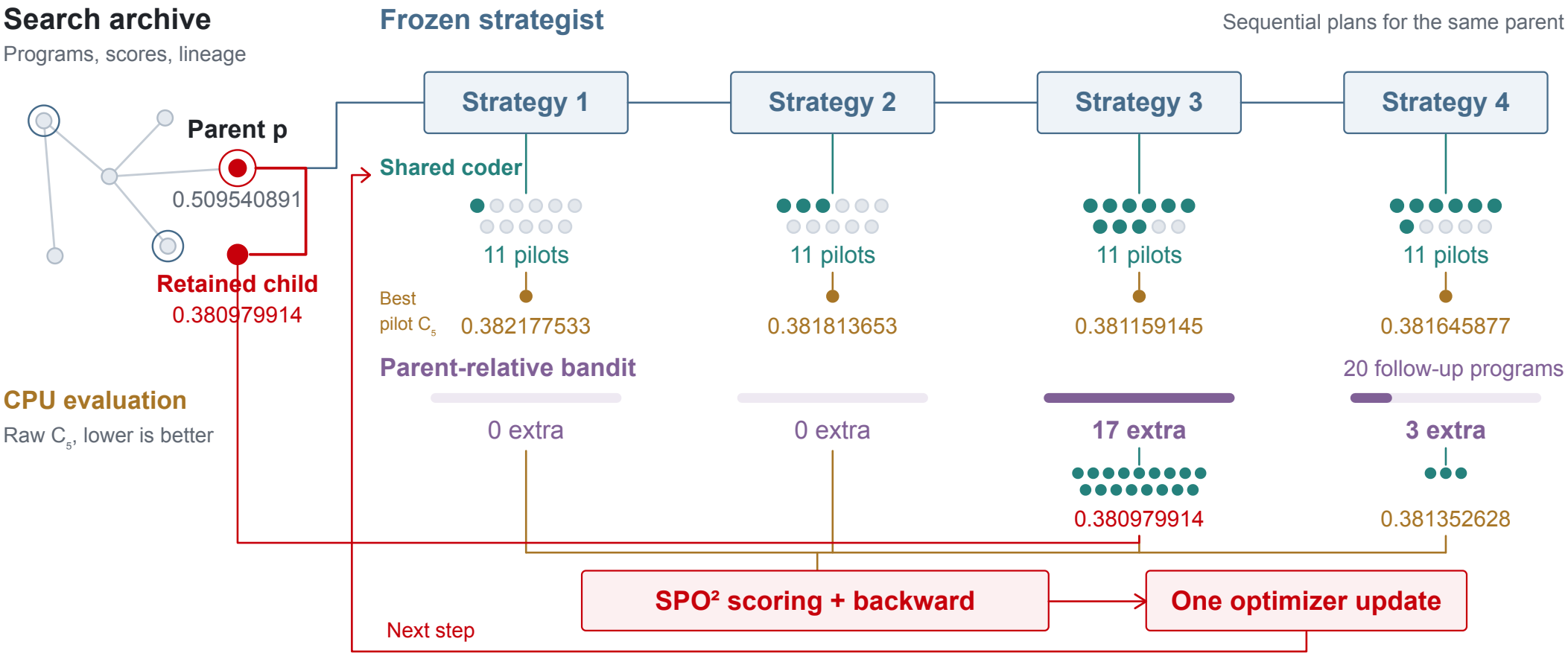
Equal sampling continues after early results indicate that some directions are more promising. The remaining budget cannot shift toward those directions.

04 **Group-relative training can discard useful reward information**

When all programs in a group receive the same reward, their group-relative advantages are zero. This removes the reward-based update even if that reward improves on earlier results.

Technical Solution: Hierarchical Discovery Pipeline

Retain children and update the shared coder



Selected children enter the archive. One optimizer update prepares the coder for the next step.

Technical Solution: Allocation and Policy Training

Parent-relative bandit allocation

- Generate H pilot programs for each of S strategies.
- Estimate each plan's chance of improving on its parent and the size of successful gains.
- Allocate B follow-ups by their expected effect on the best sampled reward, accounting for uncertainty.

$$Z_{k,j} = \max(R_{k,j} - R_p, 0)$$

$R_{k,j}$: reward of pilot j for strategy k. R_p: parent reward.
$Z_{k,j}$: improvement over the parent, set to zero for invalid outputs.

$$a_k \in \mathbb{Z}_{\geq 0}, \quad \sum_k a_k = B \quad N_{\text{parent}} = SH + B$$

a_k: follow-ups for strategy k.
N_{parent}: total planned programs for that parent.

Historical-baseline training (SPO²)

- Higher rewards receive exponentially larger weights.
- Compute every advantage against the same moving average of earlier transformed rewards.

$$G(r) = e^{\beta r}, \quad \beta > 0$$

$$A_{t,i} = \frac{1}{\beta}\left(\frac{e^{\beta r_{t,i}}}{v_{t-1}} - 1\right)$$

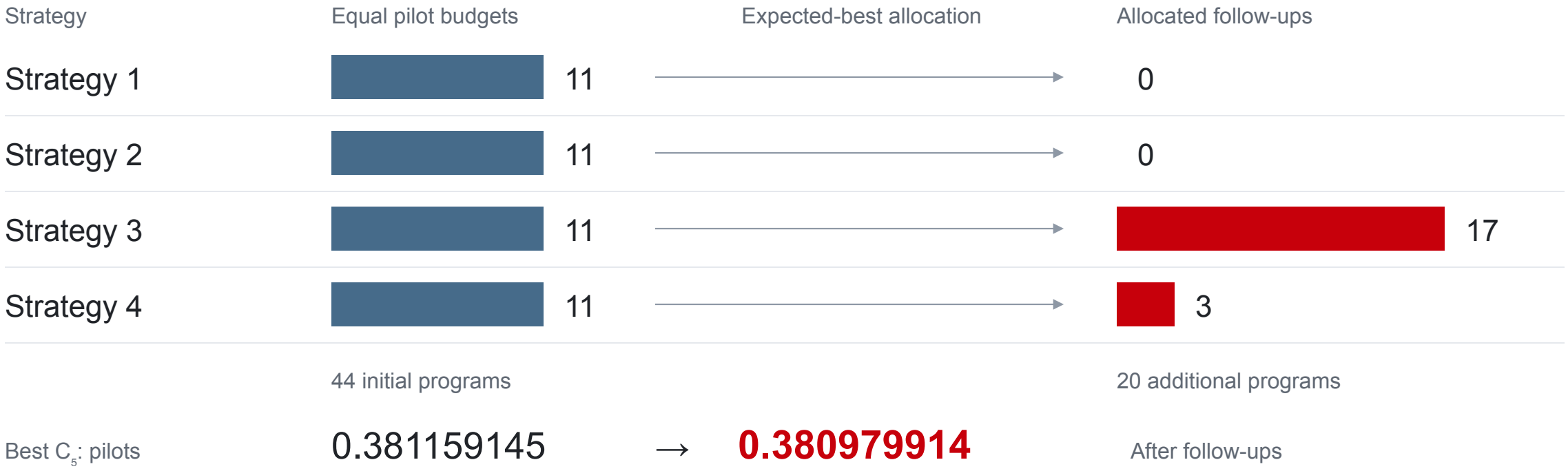
v_{t-1}: moving average of earlier G(r). β: strength of reward shaping.
$r_{t,i}$: reward. $A_{t,i}$: training weight for rollout i at step t.

Positive A reinforces a program. Negative A discourages it. A tied group can still receive a nonzero weight relative to earlier rewards. Hold β and the baseline fixed within the step, then update the baseline.

R and r denote larger-is-better training rewards. Scientific plots show the raw task score, minimized for the Erdős problem.

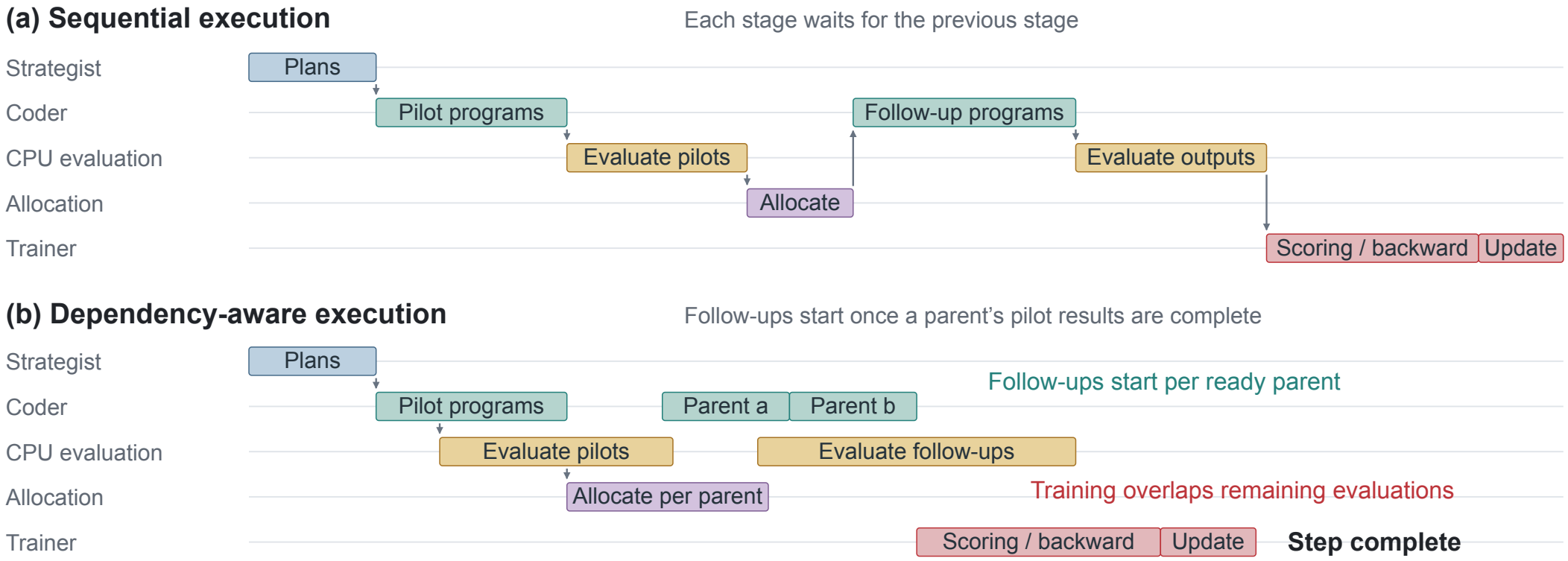
Observed Allocation: Equal Pilots, Targeted Follow-ups

One parent has a budget of 64 programs. Generate 11 pilots for each of four strategies, then use their results to allocate the remaining 20 programs.



Erdős minimum-overlap task. C_5 is the evaluated bound, lower is better.

Technical Solution: Dependency-Aware Execution



Evaluate programs as they arrive. Once generation ends, train on evaluated programs while CPU evaluation continues. **Keep parameters fixed until the step's final optimizer update.**

Distinguishing Features of the Idea

- **Separate planning from learning:** the strategist proposes plans with fixed weights. Only the shared coder learns from evaluated programs.
- **Compare each plan with its parent:** estimate both improvement probability and gain size, so rare large gains enter the allocation decision.
- **Allocate after observing the pilots:** use their results to choose follow-up counts while keeping the total program budget fixed.
- **Use one historical baseline across groups:** compare outputs with earlier rewards. Larger policy changes reduce the weight of older rewards.
- **Account for related archive states:** update ancestor expansion counts and avoid selecting a solution with its own descendant in the same batch.
- **Overlap work without changing the policy:** train on evaluated outputs while other evaluations continue, then apply one final update.

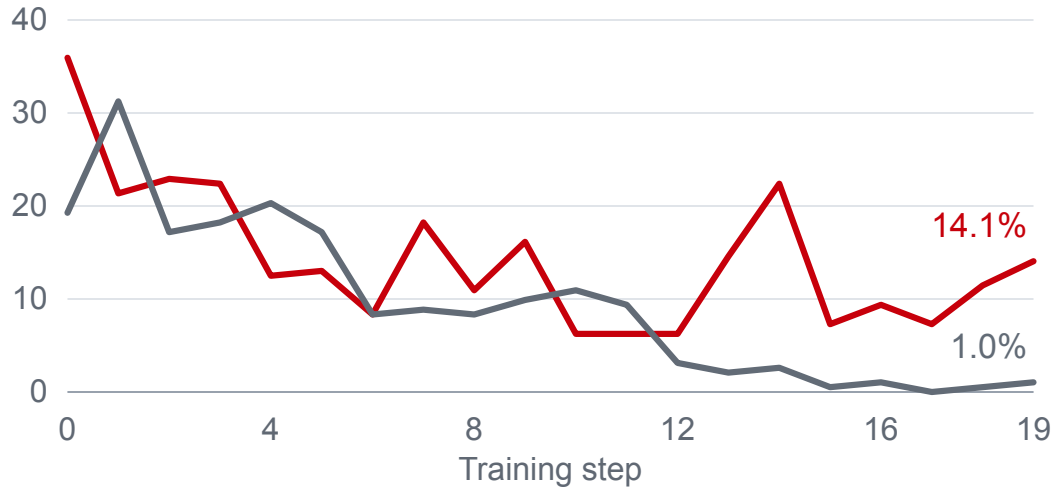
Advantageous Effects: Construction-Level Exploration

Erdős task: train a Qwen3-8B coder on task + parent context, with or without an added frozen-strategist plan.

With strategist Without strategist

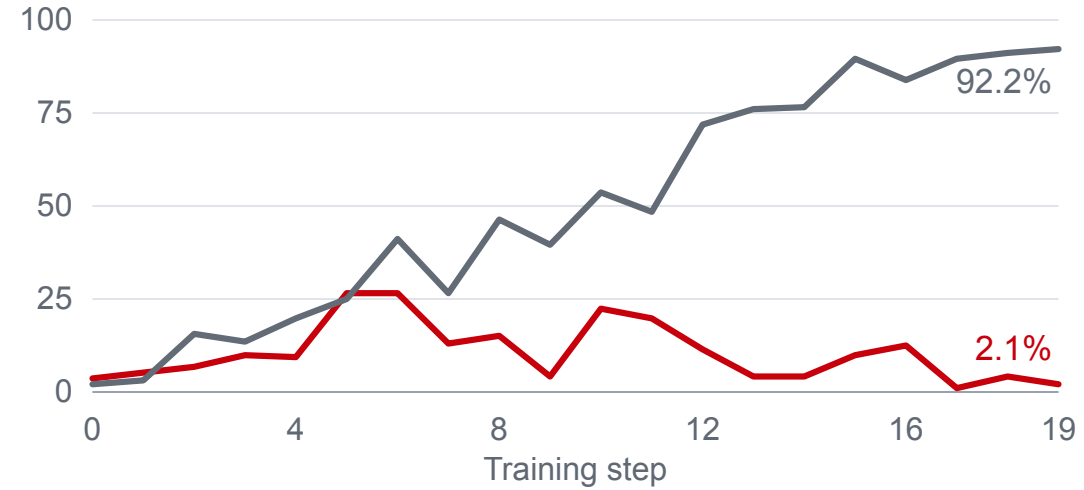
192 coder attempts per step in both runs

Distinct non-parent outputs (%)



Left: unique valid constructions different from every selected parent, counted once per step.

Parent-copy rate (%)



Right: valid constructions equivalent to any selected parent, including repeated copies.

Strategy conditioning sustains distinct outputs late in training. Direct generation mostly copies parents.

Experiments: Coder Learning and Reward Design

These runs compare binary success and score-based rewards, and group-relative and historical-baseline training.

Coder learning with a frozen strategist

R1-Distill-Qwen-32B strategist
Qwen3-Coder-30B-A3B-Instruct coder, no bandit

Coder update	Programs	Reward > 0	Best C_5 ↓
RL off	4.2k	70.38%	0.381009
Binary-PPO	2.3k	68.61%	0.381086
SPO²	4.8k	63.74%	0.380971

Why score-based rewards

Two valid programs can return different C_5 bounds. Binary success gives them the same reward. A score-based reward retains this difference.

Direct Qwen3-8B coder

No strategist
Compare group-relative and historical-baseline learning

Coder update	Programs	Reward > 0	Best C_5 ↓
Entropic-GRPO	12.2k	85.23%	0.380988
SPO²	4.6k	78.39%	0.380980

Why a historical baseline

When all rewards within a group are equal, group-relative advantages are zero. A historical baseline can still assign a nonzero advantage relative to past rewards.

Experiments: Planning and Coding Models

A higher reward-positive rate did not necessarily correspond to a lower C_5 bound.

Same Qwen3-8B coder and entropic-GRPO, different frozen strategists

Frozen strategist	Programs	Reward > 0	Best C_5 ↓
Qwen3-8B	6.6k	64.02%	0.381022
R1-Distill-Qwen-32B	3.1k	43.45%	0.380919

The R1 planner run reached a lower C_5 bound despite a lower positive-reward rate.

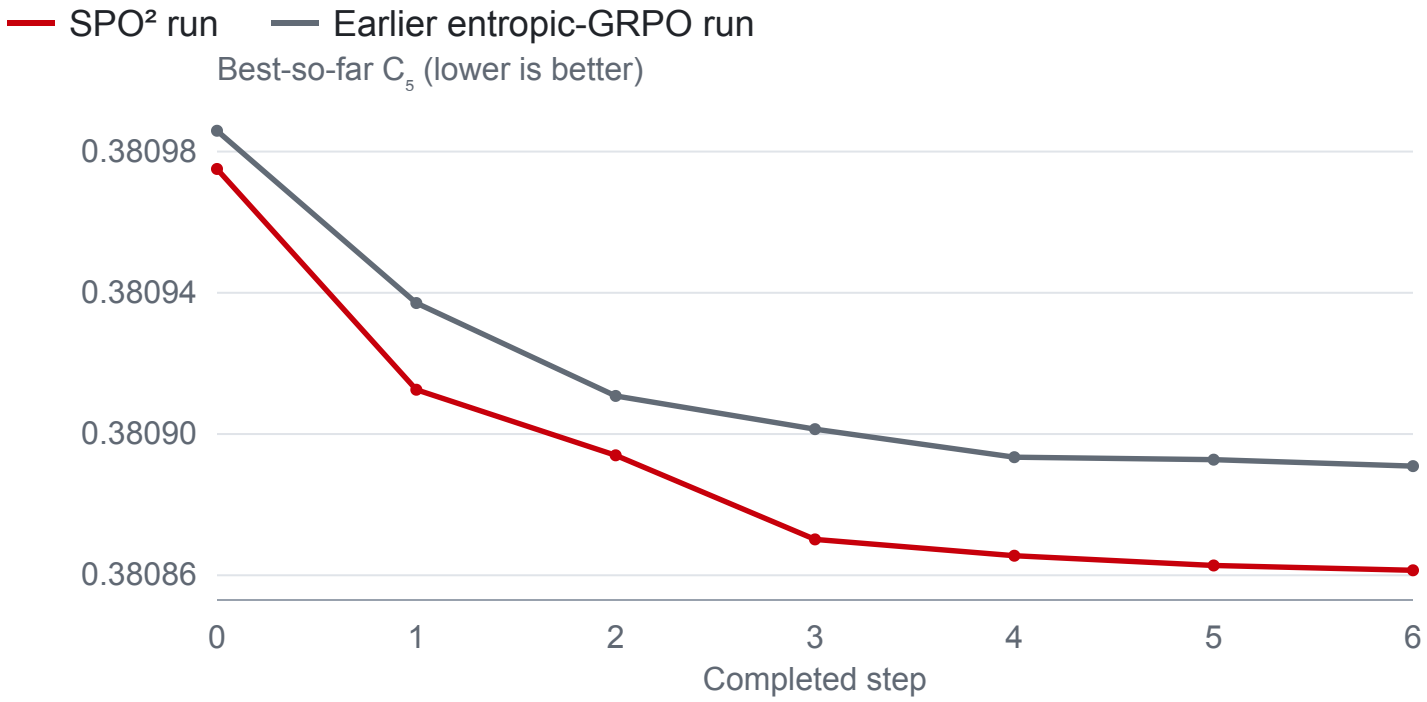
Same R1-Distill-Qwen-32B strategist, different coder configurations

Coder and update	Programs	Reward > 0	Best C_5 ↓
Qwen3-Coder-30B (SPO²)	4.8k	63.74%	0.380971
Qwen3-8B (Entropic-GRPO)	3.1k	43.45%	0.380919

The Qwen3-8B run reached a lower bound than the Qwen3-Coder-30B run, but its RL objective also differed.

Experiments: Two Runs of the Full Pipeline

Same Qwen3.8-27B coder and frozen R1-0528-Qwen3-8B strategist. Both runs use pilots and bandit follow-ups.



Observed bounds at step 6

Earlier entropic-GRPO

0.380890913

544 programs per step, 24 follow-ups per parent

SPO²

0.380861410

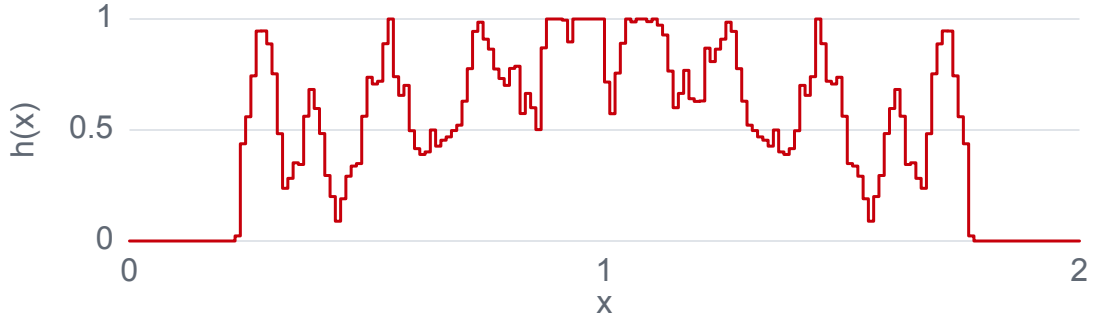
512 programs per step, 20 follow-ups per parent

The runs also differ in context limits, reasoning settings, and rollout budgets. This comparison therefore cannot isolate the effect of the RL objective or allocator.

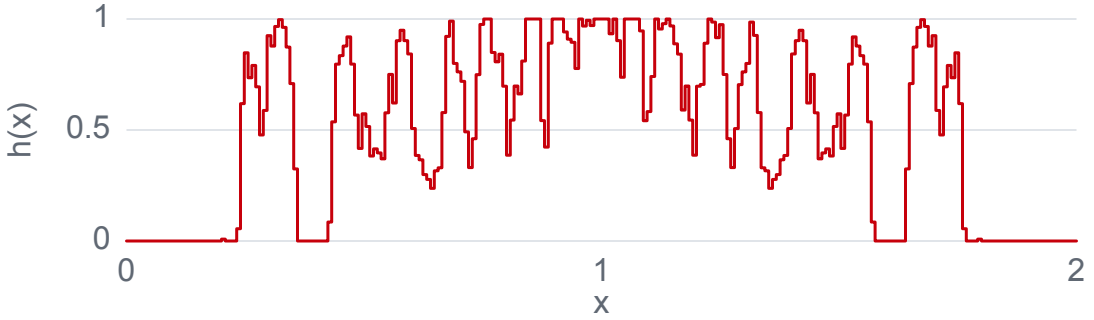
Advantageous Effects: Saved Construction Evolution

Each plot shows the step's best h(x), with n equal-width pieces. Lower C_5 means less peak overlap with shifted 1 − h(x).

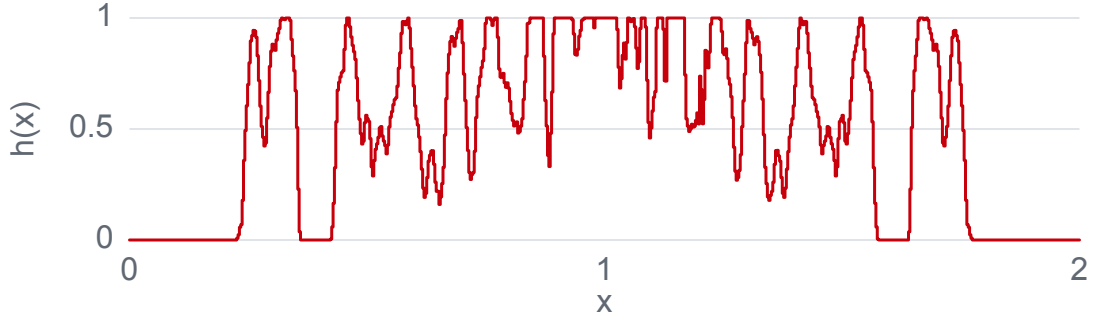
Step 0 n = 180 C_5 = 0.380975069



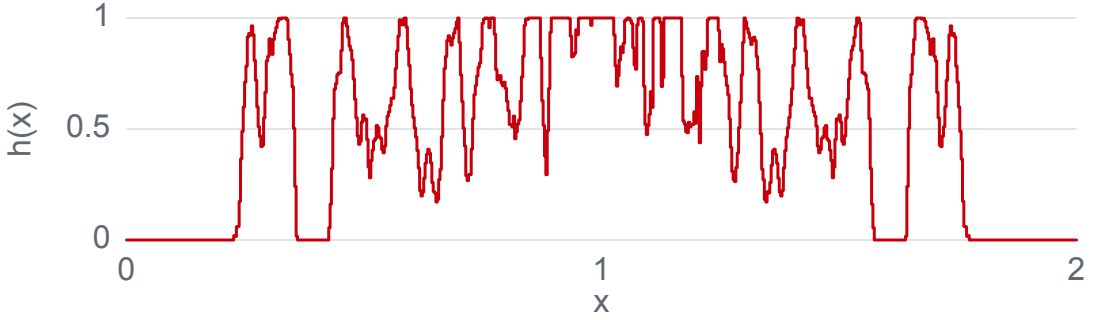
Step 3 n = 250 C_5 = 0.380870161



Step 7 n = 700 C_5 = 0.380861062

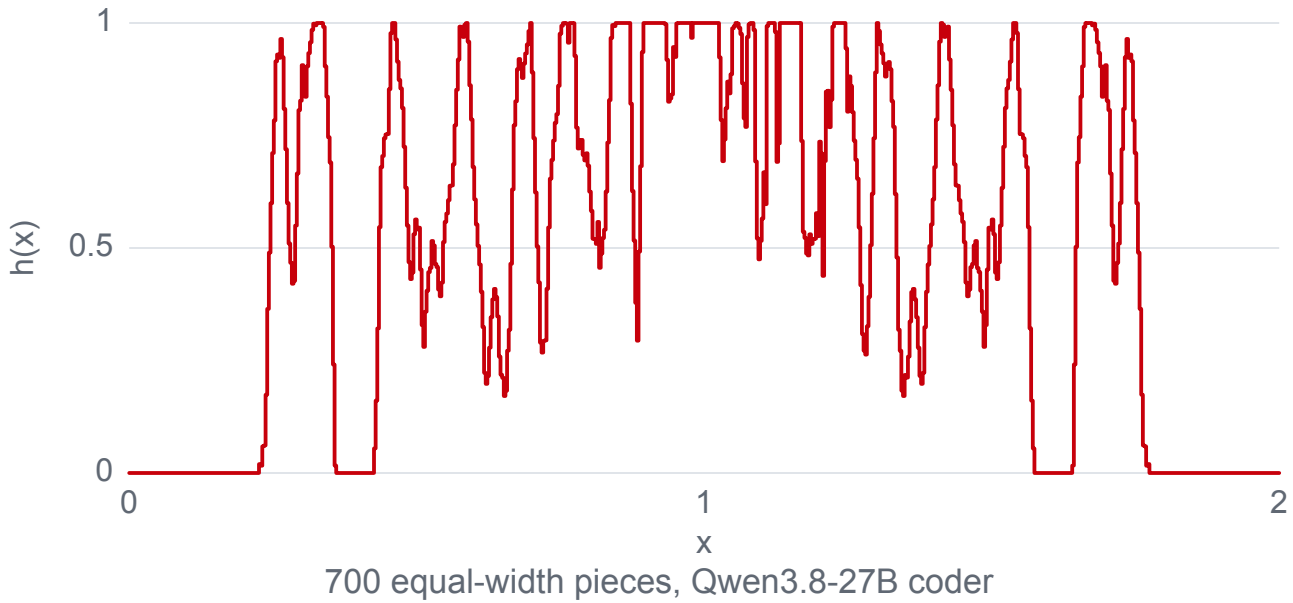


Step 10 n = 700 C_5 = 0.380860634



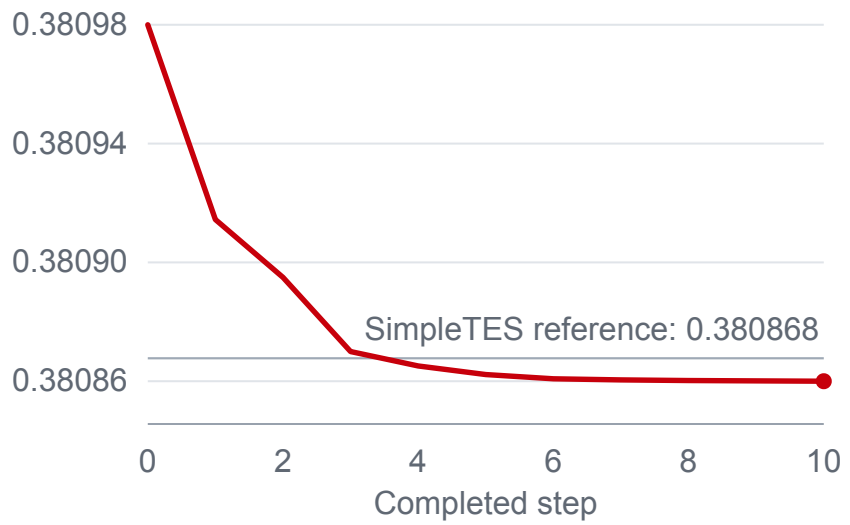
Advantageous Effects: Evaluated Scientific Result

Best saved construction



700 equal-width pieces, Qwen3.8-27B coder

Best-so-far C_5 bound



$C_5 \leq 0.380860634340$

The full pipeline found a construction below the historical reference of 0.380868. Re-evaluating its saved array reproduces the bound shown here.

Value Statement for Obtaining a Patent

Aspect	Proposed patent value
Technical contribution	A frozen LLM proposes plans. Pilot outcomes allocate further programs, while evaluated programs train a shared LLM and update the search archive.
Potential use	Tasks where a generated program returns a candidate and an evaluator checks its constraints and score.
Detection	Prompt records show the two model roles. Allocation and training logs show how pilot outcomes set budgets, how the historical baseline changes, and when updates occur.
Alternative designs	Uniform allocation and group-relative learning are alternatives. Isolating their effects requires matched-setting experiments.
Standards relevance	No direct mandatory standards relevance identified in the supplied materials.

Thank you

Hierarchical scientific discovery

Technical solution and experimental evidence